I'll spare you the output of terrible puns; take it on trust that all of the above code has legal syntax and is bug-free. If any of those expressions are confusing (perhaps you're wondering why I had to wrap the string concatenation of `'my' . 'Func` in parentheses), see Chapter 8, "Strings."

Variable function calls are not without their problems though. Let's take a look at one of PHP's oddities: not everything that looks like a function is actually a function. For example, the language construct `print` is used extensively throughout this book. I have deliberately avoided using parentheses with `print` despite the fact that they are legal syntax. `print('hello world');` does exactly the same thing as `print 'hello world';`. Here's what happens if we try to "call" `print` as we called `myFunc` above:

```
$msg = 'Important message' . PHP_EOL;
print($msg);

$funcName = 'print';
$funcName($msg);

Output:
Important message
Fatal error: Uncaught Error: Call to undefined function print()
```

The variable function call used on line 7 forces PHP to go and find a function called `print` and… there isn't one. As stated previously it is a language construct that happens to permit a function-like syntax. This problem exists for all other language constructs (`echo`, `require`, `list`, and so on). I recommend that you avoid using parentheses for language constructs in your code to avoid confusion.

Another issue with variable functions is one of debugging and analysis. Consider the following:

```
$funcName = 'wrongFunc';
$funcName();

function myFunc (): void
{
    print "Hello from myFunc" . PHP_EOL;
}
```